RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

Implementation Solution Guide

Version 1.0.0 · 2026-06-27

Contents

Executive Summary — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 4

The problem . . . 4

The solution . . . 4

What the customer gets . . . 4

Business outcomes . . . 4

Why it matters . . . 5

1. Solution Overview — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 5

Description . . . 5

Why . . . 5

Named objects produced by this solution . . . 6

Severity tiers (canonical) . . . 6

Outcomes . . . 6

2. Requirements — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 7

R1 — A production RMM with administrative access and reporting/export . . . 7

R2 — A PSA for ticket routing . . . 7

R3 — A baseline alert export . . . 7

R4 — Defined business hours and on-call schedule . . . 7

R5 — Automation/scripting capability in the RMM . . . 8

R6 — Change control / change management . . . 8

R7 — Defined monitored estate and client scope . . . 8

R8 — Severity and disposition definitions agreed with stakeholders . . . 8

R9 — A pilot client/site for safe rollout . . . 8

R10 — Metric tooling and a reporting home . . . 9

R11 — Read-only operating posture for assessment . . . 9

3. Implementation — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 9

Phase A — Assess (baseline) . . . 9

Phase B — Classify . . . 10

Phase C — Tune thresholds . . . 10

Phase D — Automate / self-heal . . . 11

Phase E — Route . . . 11

Phase F — Validate . . . 12

Phase G — Operate . . . 12

5. Testing & Validation — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 12

Gate 1 — Noise reduced versus baseline . . . 12
Gate 2 — Percent actionable increased . . . 13
Gate 3 — No missed criticals . . . 13
Gate 4 — Alert-to-ticket mapping correct . . . 13
Secondary checks . . . 13
Sign-off . . . 13
6. Troubleshooting — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 14
Assess . . . 14
Classify . . . 14
Tune thresholds . . . 14
Automate / self-heal . . . 15
Route . . . 15
Validate . . . 15
Operate . . . 16
7. Best Practices — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 16
Measure before you touch anything . . . 16
Tune by impact, not alphabetically . . . 16
Every alert must have an owner and an action . . . 16
Always use duration and hysteresis, never bare thresholds . . . 16
Correlate root causes; suppress the dependent storm . . . 17
Suppress planned work . . . 17
Self-heal first, ticket only on failure . . . 17
Route by severity AND time-of-day . . . 17
Protect against missed criticals . . . 17
Make changes reversible and recorded . . . 17
Operate it as a recurring loop, not a project . . . 17
Stay vendor-agnostic in method, vendor-specific in execution . . . 18
8. Deliverables Summary — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting 18
Documentation . . . 18
Scripts (PowerShell) . . . 18
Hero image . . . 19
How the deliverables map to the loop . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

The problem

Most managed-services teams do not lack a monitoring tool — they lack a **tuned** one. Out-of-the-box RMM policies are intentionally noisy so nothing is missed, which trains technicians to ignore alerts. The result is alert fatigue, inflated ticket counts that distort capacity, and — most dangerously — real critical conditions buried under hundreds of low-value notifications. When everything alerts, nothing does, and the one alert that mattered at 2 a.m. scrolls past unread.

The solution

This is a vendor-agnostic **methodology and runbook** to tune an existing RMM so it surfaces real issues earlier and cuts noise sharply — without buying new tooling. It runs a closed measurement loop, **measure** → **classify** → **threshold** → **automate** → **re-measure**, across seven phases: **Assess (baseline)** → **Classify** → **Tune thresholds** → **Automate/self-heal** → **Route** → **Validate** → **Operate.**

The engagement establishes a hard baseline (alerts per endpoint per day, percent actionable, MTTA, MTTR), classifies every alert into a four-tier severity matrix, tunes thresholds with sustain-duration, hysteresis, and auto-close, deduplicates and correlates alert storms, routes by severity and time-of-day, maps surviving alerts into correctly-categorized PSA tickets, adds self-healing automations for common conditions, and then re-measures to **prove** the result.

What the customer gets

- A documented, repeatable tuning loop usable across **NinjaOne, Datto RMM, ConnectWise Automate**, and other RMMs.
- Eight documents (overview, requirements, global requirements, implementation, testing, best practices, troubleshooting, this summary).
- Five PowerShell tools to baseline, report, validate, and flag noisy sources — all read-only by default.
- A named-object framework (severity matrix, threshold policy set, dedup/correlation ruleset, self-heal library, routing matrix, alert-to-ticket mapping, and the before/after baseline report) so the work is auditable and operable.

Business outcomes

- **Alert volume cut by a typical 60–90%**, with a non-waivable gate ensuring **no critical is ever missed**.
- **Percent-actionable alerts up** (commonly to ≥60%), restoring technician trust in the console.
- **Lower MTTA/MTTR** and fewer phantom tickets, improving SLA performance and capacity accuracy.
- **Common conditions self-heal** before a human is involved, reclaiming technician time.
- A recurring operating cadence so the gains hold as the estate changes.

Why it matters

Tuning is the highest-leverage, lowest-cost improvement an operations team can make: no new license spend, only method. The difference between a team drowning in alerts and a team that trusts its monitoring is rarely the product — it is whether anyone ever applied this loop. This solution applies it, and proves it.

1. Solution Overview — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

Description

This solution is a vendor-agnostic **methodology and runbook** for tuning an existing Remote Monitoring & Management (RMM) platform so it surfaces real issues earlier and produces dramatically less noise. It does not install or replace an RMM; it **tunes the one already in production** — the alert taxonomy, severity matrix, thresholds, hysteresis and auto-close rules, deduplication and correlation, business-hours versus after-hours routing, the alert-to-ticket mapping into the PSA, self-healing automations for common conditions, and a measurable noise-reduction baseline.

The work follows a closed measurement loop: **measure** → **classify** → **threshold** → **automate** → **re-measure**. You begin by exporting the current alert stream and computing a baseline (alerts per endpoint per day, percent actionable, MTTA, MTTR). You then classify every alert type into a severity matrix, tune thresholds with hysteresis and auto-close so transient blips stop paging humans, deduplicate and correlate storms, route by severity and time-of-day, map the survivors cleanly into PSA tickets, and add self-heal automations that resolve common conditions before a human is ever involved. Finally you re-measure against the baseline to prove the noise dropped while actionability and detection improved.

The phase model used throughout is: **Assess (baseline)** → **Classify** → **Tune thresholds** → **Automate/self-heal** → **Route** → **Validate** → **Operate.**

Why

Most managed-services teams do not have an RMM problem — they have an **RMM tuning** problem. Out-of-the-box monitoring policies are deliberately noisy so nothing is missed, which trains technicians to ignore alerts. When everything is an alert, nothing is. The cost is real: missed critical conditions buried under thousands of low-value notifications, alert fatigue and burnout, inflated ticket counts that distort capacity planning, and SLA breaches because the one alert that mattered scrolled past at 2 a.m. alongside two hundred that did not.

Tuning is the highest-leverage, lowest-cost improvement available to an operations team because it requires no new tooling spend — only method. A disciplined tuning pass routinely cuts alert volume by **60–90%** while *increasing* the share of alerts that are genuinely actionable and reducing mean time to acknowledge (MTTA) and mean time to resolve (MTTR). The difference between a team drowning in alerts and a team that trusts its console is rarely the product; it is whether anyone ever applied this loop.

This solution is deliberately **vendor-agnostic**. The tuning method, the policy standards, the severity matrix, and the metrics are identical across **NinjaOne, Datto RMM, and ConnectWise Automate** (and others). Where a concept maps to a product feature, named examples are given, but product-specific click

paths are intentionally deferred to the chosen RMM's own documentation, which changes faster than any runbook.

Named objects produced by this solution

Every object below is named – – and is referenced verbatim throughout the implementation, testing, and operations content:

- **Alert Inventory & Baseline – RMM Estate – Assess** — the exported, parsed alert dataset plus the computed baseline metrics.
- **Severity & Taxonomy Matrix – Alert Classes – Classify** — the canonical mapping of every alert type to a severity tier and disposition.
- **Threshold & Hysteresis Policy Set – Monitor Conditions – Tune** — the tuned monitor thresholds, durations, hysteresis bands, and auto-close rules.
- **Dedup & Correlation Ruleset – Alert Stream – Tune** — the suppression, deduplication, and correlation rules that collapse storms.
- **Self-Heal Automation Library – Common Conditions – Automate** — the catalog of remediation/self-heal scripts mapped to conditions.
- **Routing & Escalation Matrix – Business Hours / On-Call – Route** — severity-by-time routing, on-call assignment, and escalation paths.
- **Alert-to-Ticket Mapping – PSA Integration – Route** — the rules that turn surviving alerts into correctly-categorized PSA tickets.
- **Tuning Baseline Report – Noise & Actionability – Validate** — the before/after report proving noise reduction and improved actionability.

Severity tiers (canonical)

The **Severity & Taxonomy Matrix – Alert Classes – Classify** uses four tiers, reused verbatim everywhere:

- **SEV-1 Critical** — service down or imminent data loss; pages on-call 24×7; auto-creates a high-priority PSA ticket.
- **SEV-2 High** — degraded or at-risk condition needing same-business-day action; ticket during business hours, batched after hours.
- **SEV-3 Warning** — informational trend or soft threshold; ticket only if it persists or recurs; never pages.
- **SEV-4 Auto / Self-Heal** — known condition with a deterministic fix; remediation runs automatically and a ticket is created **only** if the self-heal fails.

Outcomes

On completion the operations team has a tuned RMM whose alert volume is measurably lower, whose surviving alerts are overwhelmingly actionable, whose criticals are never lost, and whose common conditions resolve themselves — all proven by the **Tuning Baseline Report – Noise & Actionability –**

Validate against the original **Alert Inventory & Baseline – RMM Estate – Assess**.

2. Requirements — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

This solution **tunes an RMM that is already deployed and collecting data**. The requirements below are what must be true before the tuning loop can run. For each, the Description states what is needed, **Why** explains why it matters to tuning, **Verify** gives a concrete check, and **Implement-if-missing** gives the remediation so the requirement is never merely assumed. Category-wide prerequisites common to all Managed solutions are in `global-requirements-managed.md`.

R1 — A production RMM with administrative access and reporting/export

Description. An RMM platform (e.g. NinjaOne, Datto RMM, ConnectWise Automate) in production with at least 30 days of alert history, an administrator account, and the ability to export or report on the alert/event stream (CSV, API, or scheduled report). **Why.** Tuning is a data exercise. Without admin rights you cannot edit monitor policies, thresholds, or routing; without export you cannot compute the baseline that every later phase is measured against. **Verify.** Sign in as an administrator; confirm you can edit a monitor policy and produce an alert export covering the last 30 days. **Implement-if-missing.** Request an admin role from the RMM owner; if history is shorter than 30 days, enable alert/event logging and reporting now and defer the **Assess** phase until at least 14 days of representative data exist.

R2 — A PSA for ticket routing

Description. A Professional Services Automation / ticketing platform (e.g. ConnectWise PSA, Autotask, Halo, Freshservice) integrated, or integratable, with the RMM. **Why.** The value of tuning is realized when surviving alerts become correctly-categorized tickets. Without a PSA target there is nowhere to route the **Alert-to-Ticket Mapping – PSA Integration – Route**. **Verify.** Confirm the RMM-to-PSA integration is connected and that a test alert can open a ticket in the correct board/queue. **Implement-if-missing.** Enable and authenticate the native RMM↔PSA integration; if none exists, configure email-parsing or webhook ticket creation as an interim and record it as a known limitation.

R3 — A baseline alert export

Description. A complete export of the current alert/event stream for the measurement window, with timestamp, device, alert type/source, severity, and (if available) acknowledge/resolve times. **Why.** The **Alert Inventory & Baseline – RMM Estate – Assess** is computed from this export. No export, no baseline, no provable result. **Verify.** Open the export and confirm it contains per-alert timestamp, device, and type columns for the full window. **Implement-if-missing.** Build a scheduled RMM report or use the RMM API to pull the alert log; `RMMTUNE-Discovery.ps1` parses common export shapes.

R4 — Defined business hours and on-call schedule

Description. Documented business hours per client/site and a current on-call/after-hours roster. **Why.** The **Routing & Escalation Matrix – Business Hours / On-Call – Route** routes by severity *and* time-of-day; ambiguous hours produce mis-paged technicians or silent gaps. **Verify.** Confirm a written business-hours

definition (with time zone) and the active on-call schedule exist. **Implement-if-missing.** Capture business hours per client in the CMDB/PSA and stand up an on-call rotation before configuring after-hours routing.

R5 — Automation/scripting capability in the RMM

Description. The RMM's scripting/automation engine (PowerShell, Bash, or the product's native automation) enabled, with the ability to trigger a script from an alert condition. **Why.** The **Self-Heal Automation Library – Common Conditions – Automate** depends on the RMM being able to run a remediation on a monitored condition; without it, SEV-4 self-heal is impossible and noise stays high. **Verify.** Run a trivial script (e.g. write a log line) against a pilot device from the RMM, and confirm an alert can trigger a script. **Implement-if-missing.** Enable the automation module, deploy the agent script runner, and grant the automation role the rights to execute remediations.

R6 — Change control / change management

Description. A change-control process so threshold, routing, and automation changes are approved, recorded, and reversible. **Why.** Tuning edits live monitoring for production clients; an un-reviewed change can suppress a real critical. Every change must be traceable and rollback-able. **Verify.** Confirm a change record can be raised and that the prior policy state is captured before edits. **Implement-if-missing.** Adopt the change-control process in `global-requirements-managed.md`; snapshot/export every policy set before editing it.

R7 — Defined monitored estate and client scope

Description. A known, current inventory of which devices/clients are in scope, so per-endpoint metrics are meaningful. **Why.** Alerts-per-endpoint-per-day requires an accurate endpoint count; a stale count distorts the baseline and the result. **Verify.** Reconcile the RMM device count against the CMDB for the in-scope clients. **Implement-if-missing.** Sync the RMM inventory, decommission stale agents, and confirm the endpoint count before computing the baseline.

R8 — Severity and disposition definitions agreed with stakeholders

Description. Agreement from service-desk and account stakeholders on what SEV-1…SEV-4 mean and the response expectation for each. **Why.** The **Severity & Taxonomy Matrix – Alert Classes – Classify** is only useful if the desk acts on the tiers consistently; undefined severity is the root cause of noise. **Verify.** Confirm a signed-off severity definition (the four canonical tiers) exists and the desk has seen it. **Implement-if-missing.** Run a short workshop to ratify the four-tier model in Section 1 before classifying alerts.

R9 — A pilot client/site for safe rollout

Description. One representative client or device group designated for first application of tuned policies before fleet-wide rollout. **Why.** Tuning changes must be validated on a contained population so a mistake affects one client, not all. **Verify.** Confirm a pilot client is named and its stakeholders accept piloting. **Implement-if-missing.** Select a representative, lower-risk client; align with the pilot-client requirement in `global-requirements-managed.md`.

R10 — Metric tooling and a reporting home

Description. The ability to compute and store baseline/after metrics (alerts/endpoint/day, % actionable, MTTA, MTTR) and a documented home for the resulting reports. **Why.** The loop is only closed if before/after metrics are computed and retained; the **Tuning Baseline Report – Noise & Actionability – Validate** must live somewhere durable. **Verify.** Confirm a location (PSA, SharePoint, docs site) for the reports and that the supplied scripts can run against the exports. **Implement-if-missing.** Designate the documentation home (see `global-requirements-managed.md`); use `RMMTUNE-Report.ps1` and `RMMTUNE-Set-AlertBaseline.ps1` to produce and store the metrics.

R11 — Read-only operating posture for assessment

Description. Agreement that the **Assess** and **Classify** phases are read-only and make no live changes. **Why.** Trust in the process depends on proving you can measure without disrupting; all supplied discovery/report scripts default to read-only/`-WhatIf`. **Verify.** Confirm `RMMTUNE-Discovery.ps1` and `RMMTUNE-Report.ps1` are run before any policy edit. **Implement-if-missing.** Sequence the engagement so no threshold or routing change occurs until the baseline is signed off.

3. Implementation — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

This is the ordered runbook for the tuning loop. The phase model is **Assess (baseline)** → **Classify** → **Tune thresholds** → **Automate/self-heal** → **Route** → **Validate** → **Operate.** Steps are imperative and reference the named objects from Section 1 verbatim. Do not assume any prerequisite — confirm Section 2 (`02-requirements.md`) and the global requirements (`global-requirements-managed.md`) are met first. The **Assess** and **Classify** phases are read-only; no live policy changes occur until **Tune thresholds**.

Phase A — Assess (baseline)

Step 1 — Confirm scope and read-only posture

1. Confirm the in-scope clients/devices (R7) and the agreed measurement window (minimum 14 days, target 30).
2. Confirm with stakeholders that Assess and Classify make **no live changes**.
3. Raise a change record (GR4) covering the whole engagement, noting that policy edits begin only at Phase C.

Step 2 — Export the current alert stream

1. As an RMM administrator (R1), export the alert/event log for the window (R3) — via scheduled report, CSV, or API.
2. Ensure the export includes, at minimum: timestamp, device, alert type/source, severity, and acknowledge/resolve times if available.
3. Store the raw export in the documentation home (GR7) as the immutable source for the baseline.

Step 3 — Parse and profile the alerts

1. Run `RMMTUNE-Discovery.ps1` against the export to parse it and tabulate alert volume by type, by device, and by hour-of-day.

2. Identify the **top noise sources** — the alert types and devices generating the most volume.

3. Record the parsed profile as the first part of the **Alert Inventory & Baseline – RMM Estate – Assess**.

Step 4 — Compute the baseline metrics

1. Run `RMMTUNE-Set-AlertBaseline.ps1` (with `-WhatIf` first to preview, then commit) to compute and store: **alerts/endpoint/day**, **% actionable**, **MTTA**, and **MTTR**.

2. Establish the in-scope endpoint count from R7/GR3 so the per-endpoint figure is accurate.

3. Finalize the **Alert Inventory & Baseline – RMM Estate – Assess** and have stakeholders sign it off. This is the number every later phase is measured against.

Phase B — Classify

Step 5 — Build the severity & taxonomy matrix

1. Using the canonical four tiers (SEV-1 Critical, SEV-2 High, SEV-3 Warning, SEV-4 Auto/Self-Heal) from Section 1, classify **every** alert type found in Step 3.

2. For each alert type record: severity tier, whether it is actionable, the expected disposition (page / ticket / batch / self-heal / suppress), and the responsible queue.

3. Flag alert types that are non-actionable or pure noise for suppression in Phase C.

4. Save this as the **Severity & Taxonomy Matrix – Alert Classes – Classify** in the documentation home.

Step 6 — Set actionability targets

1. From the baseline % actionable, set the target uplift (e.g. raise from 15% to ≥60%).

2. Identify which classifications drive the largest noise reduction so Phase C is sequenced by impact.

Phase C — Tune thresholds

Step 7 — Snapshot existing policies (rollback point)

1. Before editing anything, export/snapshot each monitor policy set you will change and attach it to the change record (GR4). This is the rollback point.

Step 8 — Apply thresholds, duration, and hysteresis

1. For each noisy SEV-2/SEV-3 condition, raise the trigger threshold and add a **sustain duration** (the condition must persist N minutes/samples before alerting) so transient spikes are ignored.

2. Add **hysteresis** — separate the alert (raise) threshold from the clear (reset) threshold so a value oscillating around one number does not flap. Example: alert when disk free < 10% sustained 15 min; clear only when disk free ≥ 15%.

3. Add **auto-close** rules so a condition that self-recovers closes its own alert without human action.

4. Record all changes as the **Threshold & Hysteresis Policy Set – Monitor Conditions – Tune**, referencing the vendor's docs for the exact click path (thresholds and hysteresis exist in NinjaOne condition policies, Datto RMM monitors, and ConnectWise Automate internal monitors — the concept is identical; the UI differs).

Step 9 — Deduplicate and correlate

1. Configure deduplication so repeated identical alerts from the same device collapse into one (with a count), rather than one per occurrence.

2. Configure correlation so a single root event (e.g. a site going offline) suppresses the dependent storm (every endpoint at that site) — alert on the parent, suppress the children.

3. Add maintenance-window suppression so planned reboots/patching do not alert.

4. Record these as the **Dedup & Correlation Ruleset – Alert Stream – Tune**.

Phase D — Automate / self-heal

Step 10 — Build the self-heal automation library

1. For each SEV-4 condition with a known deterministic fix (e.g. restart a hung service, clear a temp directory at high disk, restart the print spooler, re-register an offline agent), author or map a remediation script.

2. Configure each so the RMM runs the remediation automatically on the condition, and only creates a ticket **if the self-heal fails** (verify-then-act).

3. Test each remediation on the pilot client (R9/GR6) before fleet enablement.

4. Catalog these as the **Self-Heal Automation Library – Common Conditions – Automate**, naming each automation by the condition it resolves.

Step 11 — Wire self-heal outcomes back to severity

1. Update the **Severity & Taxonomy Matrix – Alert Classes – Classify** so conditions now covered by self-heal are marked SEV-4, and confirm they no longer page on success.

Phase E — Route

Step 12 — Build the routing & escalation matrix

1. Using business hours and the on-call roster (R4), define routing by **severity × time-of-day**: SEV-1 pages on-call 24×7; SEV-2 tickets during business hours and batches after hours; SEV-3 tickets only on persistence/recurrence; SEV-4 self-heals silently.

2. Define escalation paths and timers (e.g. SEV-1 unacknowledged in 15 min escalates to the next on-call).

3. Save as the **Routing & Escalation Matrix – Business Hours / On-Call – Route**.

Step 13 — Map alerts to PSA tickets

1. Configure the RMM→PSA integration (R2) so each surviving alert opens a ticket on the **correct board/queue**, with the right priority and category derived from the severity matrix.

2. Ensure deduplicated/correlated alerts produce **one** ticket, not many, and that auto-closed alerts close their tickets.

3. Save as the **Alert-to-Ticket Mapping – PSA Integration – Route**.

Phase F — Validate

Step 14 — Pilot and re-measure

1. Apply the tuned policy set to the pilot client first; let it run for a representative window.

2. Run `RMMTUNE-Report.ps1` and `RMMTUNE-Validation.ps1` to compare post-tune metrics against the **Alert Inventory & Baseline – RMM Estate – Assess** and produce PASS/FAIL against the gates in `05-testing.md`.

3. Produce the **Tuning Baseline Report – Noise & Actionability – Validate** showing before/after for noise, % actionable, MTTA, MTTR, and missed-critical count.

4. Only on PASS, roll the tuned policies to the remaining in-scope clients.

Phase G — Operate

Step 15 — Establish the recurring tuning cadence

1. Schedule a recurring (e.g. monthly) re-run of the loop: export → `RMMTUNE-Report.ps1` → review drift → re-tune.

2. Use `RMMTUNE-Remediation.ps1` (read-only/`-WhatIf` by default) to flag newly-noisy sources between full passes.

3. Update the named objects and the change record whenever policies change, keeping the documentation home (GR7) authoritative.

5. Testing & Validation — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

Validation proves the tuning loop worked: noise fell, actionability rose, nothing critical was missed, and alerts became correctly-routed tickets. Every gate compares the post-tune state against the **Alert Inventory & Baseline – RMM Estate – Assess** and is captured in the **Tuning Baseline Report – Noise & Actionability – Validate**. Run `RMMTUNE-Report.ps1` to produce the metrics and `RMMTUNE-Validation.ps1` to emit PASS/FAIL.

Each gate lists what to test, how to verify, and the pass criterion. Do not declare success on any single gate — **all four** must pass before fleet-wide rollout (per Implementation Step 14).

Gate 1 — Noise reduced versus baseline

Test. Compare alerts/endpoint/day after tuning against the baseline figure. **Verify.** Run `RMMTUNE-Report.ps1` over the post-tune window; compare to the baseline stored by `RMMTUNE-Set-AlertBaseline.ps1`. **Pass criterion.** Alerts/endpoint/day reduced by **at least the agreed target** (typical target ≥ 50%, common result 60–90%). `RMMTUNE-Validation.ps1` returns PASS for the noise metric.

Gate 2 — Percent actionable increased

Test. Compare the share of alerts that resulted in (or warranted) action, before versus after. **Verify.** Using the severity matrix dispositions and ticket outcomes, compute % actionable post-tune; compare to baseline. **Pass criterion.** % actionable **rose** to at least the target set in Implementation Step 6 (e.g. ≥ 60%). PASS from `RMMTUNE-Validation.ps1`.

Gate 3 — No missed criticals

Test. Confirm that every SEV-1/SEV-2 condition that occurred in the validation window was alerted, routed, and acknowledged — i.e. tuning did not suppress a real critical. **Verify.** Cross-check known incidents / client-reported issues in the window against the alert and ticket records; confirm each critical produced a timely alert and ticket. Review the **Threshold & Hysteresis Policy Set – Monitor Conditions – Tune** and **Dedup & Correlation Ruleset – Alert Stream – Tune** for any suppression that could hide a critical. **Pass criterion. Zero** missed criticals. This gate is mandatory and cannot be waived — any miss fails the whole validation and requires re-tuning the offending policy.

Gate 4 — Alert-to-ticket mapping correct

Test. Confirm surviving alerts open tickets on the correct PSA board/queue, at the correct priority/category, with dedup/correlation collapsing storms into single tickets and auto-close closing them. **Verify.** Inject or observe representative alerts of each severity; trace each through the **Alert-to-Ticket Mapping – PSA Integration – Route** into the PSA. Confirm one root event yields one ticket, not a storm. **Pass criterion.** Every tested severity routes to the correct board/priority; no duplicate-storm tickets; auto-closed alerts close their tickets. PASS.

Secondary checks

- **MTTA / MTTR trend.** Confirm mean time to acknowledge and resolve improved or held steady (`RMMTUNE-Report.ps1`).
- **Self-heal success rate.** Confirm SEV-4 automations in the **Self-Heal Automation Library – Common Conditions – Automate** resolve their conditions and only ticket on failure.
- **Routing correctness by time-of-day.** Confirm an after-hours SEV-2 batches and an after-hours SEV-1 pages, per the **Routing & Escalation Matrix – Business Hours / On-Call – Route**.

Sign-off

Attach the **Tuning Baseline Report – Noise & Actionability – Validate** (before/after for all four metrics plus missed-critical count) to the change record and obtain stakeholder sign-off before rolling tuned policies beyond the pilot client.

6. Troubleshooting — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

Common failure modes encountered while running the tuning loop, with likely cause and resolution. Symptoms are organized by phase (**Assess** → **Classify** → **Tune thresholds** → **Automate/self-heal** → **Route** → **Validate** → **Operate**). Each entry assumes nothing — verify the requirement (`02-requirements.md`, `global-requirements-managed.md`) before deeper diagnosis.

Assess

The alert export is missing timestamps, severity, or acknowledge/resolve times

Cause. The RMM report template excludes columns, or the API field set is narrow. **Resolution.** Rebuild the export from a richer report template or expand the API field selection (R3). At minimum capture timestamp, device, type, and severity; resolve times are needed for MTTR but the baseline can proceed without them with that metric flagged as unavailable.

`RMMTUNE-Discovery.ps1` *cannot parse the export*

Cause. Unexpected delimiter, header naming, or encoding from the vendor export. **Resolution.** Confirm the file is CSV/UTF-8; map the vendor's column names to the expected fields (the script accepts a column-mapping parameter). Re-run after correcting.

Alerts/endpoint/day looks implausibly high or low

Cause. Wrong in-scope endpoint count (stale agents inflate the denominator's error) or a partial export window. **Resolution.** Reconcile the device count against the CMDB (R7/GR3), remove decommissioned agents, and confirm the export covers the full window before re-running `RMMTUNE-Set-AlertBaseline.ps1`.

Classify

Many alert types have no agreed action

Cause. Severity definitions were never ratified with the desk (R8). **Resolution.** Run the severity workshop to ratify the four canonical tiers; any alert with no defined action is SEV-3 or suppressed. Update the **Severity & Taxonomy Matrix – Alert Classes – Classify**.

Tune thresholds

Alerts still flap after raising the threshold

Cause. No sustain duration or no hysteresis — the value oscillates across a single bare threshold. **Resolution.** Add a sustain duration and separate the raise/clear thresholds (hysteresis) per Implementation Step 8. Confirm auto-close is enabled so self-recovering conditions clear themselves.

A storm of identical alerts from one device persists

Cause. Deduplication not configured. **Resolution.** Enable dedup so repeated identical alerts collapse into one with a count (**Dedup & Correlation Ruleset – Alert Stream – Tune**).

A site outage still produces hundreds of endpoint alerts

Cause. No correlation/parent suppression. **Resolution.** Configure correlation to alert on the parent (site/upstream) and suppress dependent children. Verify dependency mappings are current.

Planned patching/reboots generate alerts

Cause. Maintenance windows not wired to suppression. **Resolution.** Configure maintenance-window suppression so planned work does not alert.

Automate / self-heal

A self-heal automation runs but the condition returns immediately

Cause. The remediation treats a symptom, not the root cause, or lacks a verify step. **Resolution.** Make the automation verify → act → verify; if it cannot confirm a fix, fail to a ticket rather than looping. Re-test on the pilot client (R9/GR6).

Self-heal never fires

Cause. The RMM automation module is disabled or the alert→script trigger is not wired (R5). **Resolution.** Enable the automation engine, confirm the agent runner is deployed, and bind the SEV-4 condition to its remediation.

Route

After-hours pages fire for non-critical conditions

Cause. Time-of-day routing not applied; SEV-2/SEV-3 paging after hours. **Resolution.** Apply severity × time-of-day routing so only SEV-1 pages after hours; SEV-2 batches (**Routing & Escalation Matrix – Business Hours / On-Call – Route**).

One root event creates many PSA tickets

Cause. Dedup/correlation not reflected in the alert-to-ticket mapping. **Resolution.** Ensure correlated/deduplicated alerts produce one ticket; verify auto-closed alerts close their tickets (**Alert-to-Ticket Mapping – PSA Integration – Route**).

Tickets land on the wrong board/queue or priority

Cause. Mapping rules not derived from the severity matrix. **Resolution.** Align the PSA mapping to the severity tiers and per-client board/queue (R2); re-test each severity.

Validate

`RMMTUNE-Validation.ps1` *returns FAIL on noise reduction*

Cause. Top noise sources not yet tuned, or the validation window predates the changes. **Resolution.** Confirm the validation window is fully post-change; re-tune the remaining top sources from the discovery profile and re-measure.

Gate 3 flags a missed critical

Cause. An over-aggressive threshold, suppression, or correlation hid a real SEV-1. **Resolution.** This is non-waivable. Identify the offending rule in the **Threshold & Hysteresis Policy Set – Monitor Conditions – Tune** or **Dedup & Correlation Ruleset – Alert Stream – Tune**, loosen it, re-validate, and document the correction in the change record.

Operate

Noise creeps back over weeks

Cause. Estate changed; new devices/apps introduced untuned conditions. **Resolution.** Run the recurring loop (export → `RMMTUNE-Report.ps1` → review drift → re-tune). Use `RMMTUNE-Remediation.ps1` (read-only/`-WhatIf`) to flag newly-noisy sources between full passes.

A tuned policy was changed and no one knows what it was

Cause. Change not snapshotted/recorded. **Resolution.** Restore from the pre-change snapshot attached to the change record (GR4); reinforce that every policy edit is snapshotted to the documentation home (GR7) before the change.

7. Best Practices — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

These practices keep the tuned state healthy and prevent the slow return of noise. They reinforce the phase model **Assess** → **Classify** → **Tune thresholds** → **Automate/self-heal** → **Route** → **Validate** → **Operate** and the named objects from Section 1.

Measure before you touch anything

Never tune from intuition. Always establish the **Alert Inventory & Baseline – RMM Estate – Assess** first, and re-measure after. A change you cannot prove with the **Tuning Baseline Report – Noise & Actionability – Validate** is a change you should not have made. The whole discipline is the loop: measure → classify → threshold → automate → re-measure.

Tune by impact, not alphabetically

The top handful of alert types almost always produce the majority of the noise. Sequence Phase C by the volume profile from `RMMTUNE-Discovery.ps1` so the first changes deliver the biggest drop. Resist the urge to perfect rare alerts before quieting the loud ones.

Every alert must have an owner and an action

If an alert type has no defined action, it should not page or ticket — it is SEV-3 at most, or suppressed. The **Severity & Taxonomy Matrix – Alert Classes – Classify** is the contract: an alert exists only because someone will do something specific about it.

Always use duration and hysteresis, never bare thresholds

A bare threshold flaps. Require a sustain duration before raising, and separate the raise and clear thresholds (hysteresis) so a value oscillating around the limit produces one alert, not dozens. Add auto-close so self-recovering conditions clean up after themselves. These three together — duration, hysteresis, auto-close — are the core of the **Threshold & Hysteresis Policy Set – Monitor Conditions – Tune**.

Correlate root causes; suppress the dependent storm

When a site or upstream service fails, alert on the **parent** and suppress the children. One root event should produce one ticket. Maintain the **Dedup & Correlation Ruleset – Alert Stream – Tune** as dependencies change.

Suppress planned work

Wire maintenance windows so patching, reboots, and planned changes do not alert. Unsuppressed maintenance is a leading, entirely avoidable noise source.

Self-heal first, ticket only on failure

For known deterministic conditions, run the remediation automatically (verify → act → verify) and create a ticket **only if the fix fails**. Grow the **Self-Heal Automation Library – Common Conditions – Automate** deliberately, testing each automation on the pilot client (GR6) before fleet enablement, and keep every script read-only-by-default with an explicit `-Apply`.

Route by severity AND time-of-day

A SEV-2 at 2 p.m. is a ticket; the same condition at 2 a.m. should batch, not page. Only true SEV-1 conditions page after hours. Keep the **Routing & Escalation Matrix – Business Hours / On-Call – Route** aligned with the current on-call roster and per-client business hours.

Protect against missed criticals

The single unacceptable failure mode is suppressing a real critical. Review every suppression, correlation, and raised threshold against the question "could this hide a SEV-1?" Gate 3 in testing is non-waivable for exactly this reason.

Make changes reversible and recorded

Snapshot every policy set before editing it, attach it to a change record (GR4), and keep the documentation home (GR7) authoritative. A tuned RMM with no change history is one technician away from un-tunable.

Operate it as a recurring loop, not a project

Noise creeps back as the estate changes. Schedule a recurring re-run (monthly is typical): export → `RMMTUNE-Report.ps1` → review drift → re-tune. Use `RMMTUNE-Remediation.ps1` in its default read-only/`-WhatIf` mode to flag newly-noisy sources between full passes. A downward or flat alerts/endpoint/day trend over successive runs is the proof the practice is working.

Stay vendor-agnostic in method, vendor-specific in execution

The method, severity tiers, and metrics are identical across NinjaOne, Datto RMM, ConnectWise Automate, and others. Defer only the click paths to the chosen RMM's current documentation — never hard-code a UI path into a runbook that outlives it.

8. Deliverables Summary — RMM Monitoring & Alert Tuning — Noise Reduction & Actionable Alerting

This solution ships as a documentation set, a script toolkit, and a hero image. Together they let a managed-services team run the full tuning loop **Assess** → **Classify** → **Tune thresholds** → **Automate/self-heal** → **Route** → **Validate** → **Operate** against any common RMM and prove the result.

Documentation

- **01 — Solution Overview** — description, rationale, the four canonical severity tiers, and the named objects produced.
- **02 — Requirements** — solution-specific prerequisites (R1…R11), each with Description / Why / Verify / Implement-if-missing.
- **Global Requirements (Managed)** — category-wide prerequisites (GR1…GR7) shared by all Managed solutions.
- **03 — Implementation** — the ordered, imperative runbook for the full tuning loop.
- **05 — Testing & Validation** — the four pass/fail gates (noise reduced, % actionable up, no missed criticals, alert→ticket correct) plus secondary checks.
- **07 — Best Practices** — operating guidance to keep the tuned state healthy.
- **06 — Troubleshooting** — common failure modes and fixes.
- **Executive Summary** — the one-page business case and outcomes.

Scripts (PowerShell)

All scripts default to **read-only** and carry the neutral note "Managed — read-only." Those that can change live state require an explicit `-Apply` and support `-WhatIf`.

- **RMMTUNE-Discovery.ps1** — pulls/parses an alert export and baselines alert volume by type, device, and hour. Builds the first part of the **Alert Inventory & Baseline – RMM Estate – Assess**.
- **RMMTUNE-Set-AlertBaseline.ps1** — computes and stores the alerts/endpoint/day baseline (and supporting metrics); `-WhatIf` previews. Finalizes the **Alert Inventory & Baseline – RMM Estate – Assess**.
- **RMMTUNE-Report.ps1** — produces the noise / actionability / MTTA / MTTR report to CSV. Drives the **Tuning Baseline Report – Noise & Actionability – Validate**.

• **RMMTUNE-Validation.ps1** — compares post-tune metrics against the baseline and emits PASS/FAIL against the testing gates.

• **RMMTUNE-Remediation.ps1** — flags noisy alert sources; read-only/`-WhatIf` by default, acts only with `-Apply`. Used in the Operate phase to catch drift.

Hero image

• **RMMTUNE-hero.png** — the solution hero image for the marketplace listing.

How the deliverables map to the loop

Phase	Primary deliverables
Assess (baseline)	`RMMTUNE-Discovery.ps1`, `RMMTUNE-Set-AlertBaseline.ps1`, **Alert Inventory & Baseline – RMM Estate – Assess**
Classify	Section 03/07 guidance, **Severity & Taxonomy Matrix – Alert Classes – Classify**
Tune thresholds	Section 03, **Threshold & Hysteresis Policy Set – Monitor Conditions – Tune**, **Dedup & Correlation Ruleset – Alert Stream – Tune**
Automate/self-heal	Section 03, **Self-Heal Automation Library – Common Conditions – Automate**
Route	Section 03, **Routing & Escalation Matrix – Business Hours / On-Call – Route**, **Alert-to-Ticket Mapping – PSA Integration – Route**
Validate	`RMMTUNE-Report.ps1`, `RMMTUNE-Validation.ps1`, **Tuning Baseline Report – Noise & Actionability – Validate**
Operate	`RMMTUNE-Remediation.ps1`, recurring re-run of the above

Total: **8 documents + 5 scripts + 1 hero image (`RMMTUNE-hero.png`).**

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

RMMTUNE-Discovery.ps1

RMMTUNE-Remediation.ps1

RMMTUNE-Report.ps1

RMMTUNE-Set-AlertBaseline.ps1

RMMTUNE-Validation.ps1